

Roteiro de Apresentacao

MediClaw — Assistente Inteligente de Apoio a Longevidade e Bem-Estar

Disciplina: Construção de APIs para Inteligência Artificial

Prazo de entrega: 30/06/2026

Estrutura Geral — sugestão de tempo para vídeo de 10 min

Bloco	Seção	Tempo
1	Introdução	~2 min
2	Arquitetura	~2 min
3	Documentação e Qualidade	~1 min
4	Pipeline de IA	~2 min
5	Segurança e Ética Médica	~1 min
6	Demonstração	~1 min
7	Conclusão	~30 seg
8	Apresentação da Equipe	~30 seg

1. INTRODUÇÃO

Problema

Profissionais de saúde e pacientes têm dificuldade em consolidar dados biométricos dispersos (peso, sono, atividade, nutrição) e obter orientações preventivas embasadas em evidências científicas de forma rápida e acessível durante o atendimento.

O que é o MediClaw

O **MediClaw** é uma plataforma web full-stack de apoio à decisão clínica (CDSS — Clinical Decision Support System) que combina:

- **Django REST Framework** como backend API
- **LLMs configuráveis** (OpenAI GPT-4o-mini ou Google Gemini) para geração de linguagem
- **RAG com ChromaDB** para respostas embasadas em literatura científica
- **Next.js** como frontend interativo

O sistema permite que médicos interajam com um assistente de IA durante atendimentos, consultando dados biométricos dos pacientes e recebendo orientações preventivas em tempo real via streaming.

Restrição Crítica — Ética Médica

A IA nunca emite diagnóstico médico ou prescrição. Toda resposta é de caráter educativo, preventivo e obrigatoriamente acompanhada de disclaimer médico — restrição implementada por

guardrails determinísticos na entrada e na saída do LLM.

Contexto Academico

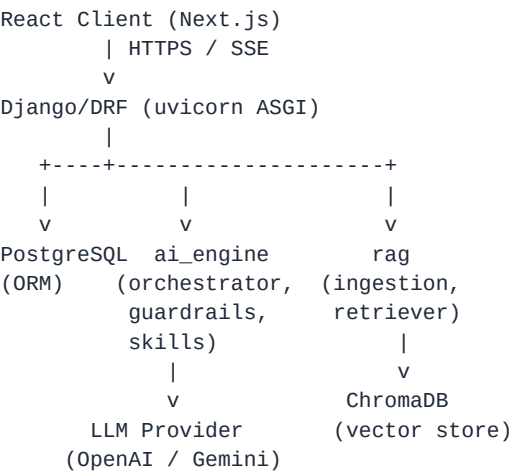
Projeto desenvolvido para a disciplina **Construcao de APIs para Inteligencia Artificial**, implementando todos os requisitos exigidos e indo alem do minimo em multiplas dimensoes.

2. ARQUITETURA

Visao Geral (Monorepo Full-Stack)

```
react-painel/      <- Frontend Next.js + TypeScript
django-api/       <- Backend Django 5.2 + DRF
```

Diagrama de Componentes



Stack Tecnica

Backend:

- Python 3.12 + Django 5.2 + DRF 3.16
- PostgreSQL 16 com psycopg[binary] 3.x
- LangChain 0.3 para orquestracao de RAG
- ChromaDB 0.5 como vector store local (MVP)
- uvicorn ASGI para suporte a streaming SSE
- structlog para logs estruturados em JSON

Frontend:

- Next.js (webpack) + TypeScript
- Comunicacao via REST e EventSource (SSE)

Infra:

- Docker + Docker Compose + DevContainer
- GitLab CI com stages de lint (Black) e test (pytest + PostgreSQL 16 real)

Modulos do Backend (Monolito Modular)

App Django	Responsabilidade
accounts	Usuarios, autenticacao JWT, registro

App Django	Responsabilidade
patients	Gestao de pacientes por medico (multi-tenancy)
health_logs	Logs biometricos: peso, sono, atividade, nutricao
conversations	Historico de chat, mensagens, streaming SSE
ai_engine	Orquestrador, prompts, guardrails, skills, providers
rag	Ingestao de documentos, vector store, retrieval
audit	ActivityLog, metricas internas
common	Excecoes, renderers, middlewares, permissoes

Decisoes Arquiteturais (ADRs)

- **ADR-01:** Django + DRF — maturidade, admin embutido para KB, ecossistema Python alinhado com IA
- **ADR-02:** PostgreSQL 16 — JSONB para metadados, caminho direto para pgvector na fase 2
- **ADR-03:** ChromaDB local — zero infra externa no MVP, abstração LangChain facilita migracao futura
- **ADR-04:** JWT stateless — sem Redis no MVP, access 30min + refresh 1 dia
- **ADR-05:** SSE (Server-Sent Events) — nativo no browser, unidirecional suficiente para chat
- **ADR-06:** Provider-agnostico de LLM — sem lock-in, troca via variavel de ambiente
- **ADR-07:** LangChain — splitters, loaders, retrievers prontos para RAG
- **ADR-08:** Guardrails deterministicos — latencia zero, auditaveis, sem custo de chamada LLM

3. DOCUMENTACAO E QUALIDADE DE CODIGO

Padrao BMAD

Toda a documentacao do projeto foi gerada seguindo o padrao **BMAD (Business, Model, Architecture, Data)**, organizada na pasta **specs/**:

- **PRD.md** — Requisitos de produto e regras de negocio
- **ARCHITECTURE.md** — Decisoes arquiteturais (ADRs), stack detalhada, fluxo de dados, schema de banco
- **PROJECT-CONTEXT.md** — Contexto completo para agentes de IA e desenvolvedores
- **TASKS.md** — Epicos e tarefas de implementacao sequenciais

Esse padrao garante que tanto desenvolvedores humanos quanto agentes de IA tenham o contexto necessario para trabalhar no projeto com coerencia.

Alternancia de Modelos — Padrao Provider

O sistema implementa um **padrao provider** que permite alternar entre modelos de linguagem sem nenhuma mudanca de codigo, apenas via variavel de ambiente:

```
LLM_PROVIDER="openai"      # Usa GPT-4o-mini + text-embedding-3-small
LLM_PROVIDER="gemini"      # Usa Gemini Flash
```

A abstracao e definida em **apps/ai_engine/providers/base.py** como um Protocol Python, com implementacoes independentes:

- **OpenAIProvider** — stream, complete, complete_json via SDK OpenAI >=1.30

• **GeminiProvider** — stream, complete, complete_json via SDK Google Generative AI

Isso evita lock-in de fornecedor e permite comparar custo e qualidade entre provedores sem refatoracao.

Suite de Testes

Camada	Framework	Quantidade
Backend (total)	pytest + pytest-django	137 testes
— Guardrails	urgencia, diagnostico, prescricao, gibberish, output LLM	22 testes
— Outros	Orquestrador, skills, RAG, autenticacao	115 testes
Frontend	Vitest + Testing Library	~40 testes
TOTAL		~177 testes

Os testes de backend rodam contra PostgreSQL 16 real (nao SQLite), garantindo fidelidade ao ambiente de producao.

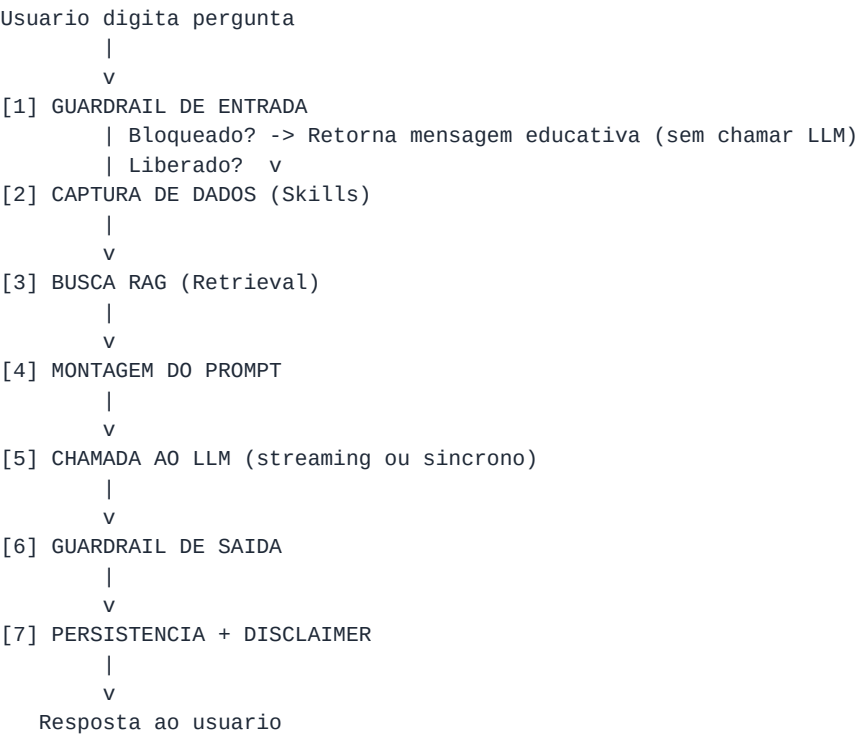
Outras Ferramentas de Qualidade

- **Black + pre-commit** — formatacao automatica em cada commit
- **Swagger UI** (GET /swagger/) e **ReDoc** (GET /redoc/) integrados via drf-yasg
- **Healthcheck** (GET /health/) — verifica Postgres e ChromaDB em tempo real
- **Conventional Commits** — historico de git semantico

4. PIPELINE DE IA

O coracao do sistema e o orquestrador em `apps/ai_engine/orchestrator.py`, que implementa um pipeline sequencial com 7 etapas distintas.

Fluxo Completo do Pipeline



Detalhamento de Cada Etapa

[1] Guardrail de Entrada (`guardrails.check_input`)

Filtros determinísticos por keywords e regex aplicados ao prompt do usuário *antes* de qualquer chamada ao LLM. Detecta e bloqueia:

- Pedidos de **diagnostico medico** (ex: "voce tem", "o diagnostico e")
- Pedidos de **presricao** (ex: "tome", "dosagem de")
- Situacoes de **urgencia/emergencia** (ex: "dor no peito", "nao consigo respirar")
- **Gibberish / texto sem sentido** (protecao contra prompt injection)
- Custo zero: sem chamada de API, sem latencia adicional, 100% auditavel

[2] Captura de Dados — Skills

O orquestrador chama automaticamente as skills relevantes para enriquecer o contexto:

- **health_summary(user_id)** — agrega dados biometricos dos ultimos 7 dias (media de peso, horas de sono, atividade)
- **calculate_bmi(weight, height)** — calculo de IMC
- **convert_units(value, from, to)** — conversao de unidades de saude

[3] Busca RAG — Retrieval Augmented Generation

Busca semantica na base de conhecimento indexada (ChromaDB):

- Embedding da query com **text-embedding-3-small** (OpenAI, 1536 dimensoes)
- Recupera os **top-5 chunks** com score de similaridade ≥ 0.75
- Chunks sao trechos de documentos cientificos/diretrizes indexados pelo admin
- Garante que as respostas sao embasadas em fontes confiaveis

[4] Montagem do Prompt

Constroi o contexto completo para o LLM: `SYSTEM_PROMPT` com contexto RAG e dados de saude, historico das ultimas 6 mensagens da conversa e a pergunta atual do usuario.

[5] Chamada ao LLM — Dois Modos

- **Modo Sincrono** (POST `/api/v1/conversations/{id}/messages/`): aguarda resposta completa, retorna mensagem com `tokens_used` e metadata (citacoes)
- **Modo Streaming SSE** (GET `/api/v1/conversations/{id}/stream/`): tokens entregues token a token via EventSource com eventos tipados: token, citation, done

[6] Guardrail de Saida (`guardrails.check_output`)

Verifica a resposta gerada pelo LLM antes de entrega-la ao usuario. Se a resposta contiver diagnostico ou presricao, e interceptada e substituida por mensagem educativa padrao. Caso contrario, injeta automaticamente o **disclaimer medico** ao final.

[7] Persistencia e Disclaimer

Persiste Message(role=ASSISTANT) com `tokens_used`, `blocked_by_guardrail` e metadata. Registra evento no audit log. Disclaimer medico obrigatorio injetado em toda resposta com vies clinico.

Endpoints de IA Implementados (4 no total — minimo exigido: 2)

Endpoint	Tipo	Pipeline
POST /api/v1/conversations/{id}/messages/	Sincrono	Guardrail + Captura + RAG + LLM + Guardrail saída
GET /api/v1/conversations/{id}/stream/	Streaming SSE	Mesmo pipeline, token a token
POST /api/v1/admin/knowledge/upload/	Ingestao	Embeddings + indexacao ChromaDB
GET /api/v1/health/summary/	Skill	Agregacao biometrica + analise por IA

5. SEGURANCA E ETICA MEDICA

Autenticacao — Tokens JWT

O sistema usa **JSON Web Tokens (JWT)** via `django-rest-framework-simplejwt`:

- **Access Token:** validade de **30 minutos** — curta o suficiente para limitar exposicao
- **Refresh Token:** validade de **1 dia** — permite renovar o access sem novo login
- **Header padrao:** Authorization: Bearer <token>
- **Stateless:** sem Redis ou sessoes em servidor no MVP
- **Fluxo:** login → recebe par (access + refresh) → usa access → ao expirar, usa refresh

***Limitacao documentada:** No endpoint SSE, o token e passado via query string (?token=...) por limitacao tecnica do EventSource nativo. O risco e mitigado pelo access token de curta duracao (30min). Mitigacao futura: cookie httpOnly.*

Rate Limiting

Perfil	Limite
Anonimo	30 requisicoes/min
Usuario autenticado	60 requisicoes/min
Endpoint de chat	10 mensagens/min

Outras Camadas de Seguranca

- **Multi-tenancy:** medicos acessam apenas seus proprios pacientes (`doctor=request.user` em todos os querysets)
- **CORS restrito:** `CORS_ALLOWED_ORIGINS` com origens explicitas — sem *
- **Upload seguro:** limite de 10MB + whitelist de MIME types (PDF, TXT, MD)
- **Sem SQL injection:** Django ORM parametrizado em todos os acessos ao banco
- **Sem log de PII:** filtro customizado no LOGGING descarta conteudo de mensagens e dados biometricos

Etica Medica e LGPD

- **Guardrails eticos** bloqueiam diagnostico, prescricao e urgencia simulada (pre e pos LLM)
- **Disclaimer obrigatorio** injetado automaticamente em toda resposta clinica
- **Consentimento LGPD** explicito no cadastro (`accepted_terms_at` obrigatorio — LGPD Art. 11)
- **Cascade delete:** ao deletar usuario, todas conversas, mensagens e logs de saude sao removidos
- **Retencao limitada:** conversas retidas por 90 dias apos ultima atividade (configuravel)

- **Minimizacao de dados:** nao coleta nem registra dado alem do necessario

6. DEMONSTRACAO

Roteiro sugerido para a gravacao (sequencia de telas):

1. **Login** → mostrar JWT sendo retornado (access + refresh)
2. **Dashboard do medico** → listar pacientes
3. **Logs biometricos** → registrar peso/sono de um paciente
4. **Chat (modo streaming)** → fazer pergunta sobre saude preventiva e mostrar tokens chegando em tempo real
5. **Guardrail bloqueando** → tentar pedir diagnostico e mostrar mensagem educativa (sem chamar LLM)
6. **Swagger UI** → mostrar documentacao interativa dos endpoints em /swagger/
7. **Upload de documento** → fazer ingestao de PDF na base de conhecimento (admin)
8. **Healthcheck** → GET /health/ mostrando status de Postgres e ChromaDB

Cenarios de dado invalido (critério de avaliacao):

- Enviar payload sem campo content → mostrar VALIDATION_ERROR estruturado
- Fazer upload com MIME type incorreto → mostrar INVALID_FILE_TYPE

7. CONCLUSAO

O que o projeto entrega alem do minimo

Criterio	Minimo exigido	MediClaw
Endpoints com IA	2	4 endpoints distintos
Validacao de dados	Basica	Multipas camadas (serializers, guardrails, MIME)
Tratamento de erros	Basico	Envelope padronizado + erros SSE sem quebrar stream
Seguranca	JWT	JWT + rate limiting + CORS + multi-tenancy + guardrails
Testes	—	~177 testes (137 backend + ~40 frontend)
Documentacao	README	Swagger + ReDoc + PRD + ADRs + BMAD

Limitacoes Documentadas (Trabalho Futuro)

- Audit trail atual e stub (pass) — persistencia real planejada para fase 2
- Token SSE via query string — mitigacao: cookie httpOnly ou proxy WebSocket
- ChromaDB → pgvector no PostgreSQL (zero mudanca no contrato do retriever)
- Indexacao sincrona → Celery + Redis para documentos grandes

O MediClaw demonstra dominio avancado dos conceitos da disciplina: pipeline de IA completo com guardrails eticos, RAG, streaming SSE, provedores intercambiaveis, suite de testes robusta e documentacao de nivel profissional seguindo o padrao BMAD.

8. APRESENTACAO DA EQUIPE

Membro
Adriano Lopes de Mendonca Soares
Julio Cesar Batista Pires
Luciano Antonio Cordeiro de Sousa
Mara Joziclea Pereira
Pedro Ramos Krauze Diehl

Roteiro gerado em: 26/06/2026 | Proximo passo: gerar apresentacao.pptx a partir deste roteiro